

CYBR 437: Secure Coding

Assembly for C

Instructor: Abinash Borah

Outline

- Assembly Language Fundamentals
- Disassembling C Programs

Assembly Language

- Assembly is the human-readable version of how the computer sees programs written by us
- Compiler tools translate code we write into lower-level machine code that the computer runs
- Assembly is a human-readable version of machine code
- Assembly is lower-level than our programming languages
- For example, there may be multiple assembly instructions needed to encode a single C instruction

```
push    %rbp
mov     %rsp, %rbp
sub     $0x20, %rsp
mov     %fs:0x28, %rax
mov     %rax, -0x8(%rbp)
xor     %eax, %eax
movl    $0x14, -0x20(%rbp)
movl    $0x96, -0x1c(%rbp)
leal    -0x20(%rbp), %rax
mov     %rax, -0x18(%rbp)
leal    -0x20(%rbp), %rax
mov     %rax, -0x10(%rbp)
leal    -0x20(%rbp), %rax
mov     %rax, %rsi
```

Why Assembly?

Learning about assembly allows us to:

- Read assembly code and translate it back to C code that may have generated it
- Better understand how our program is converted into machine instructions and use this understanding to write better code
- Understand the behavior of programs even without having access to their source code (reverse engineering)

What do we see in assembly for some C code?

- Name and memory address of a function – where the code for the function starts
- Memory addresses of the instructions – sequential instructions are sequential in memory
- Assembly code – human-readable version of machine code instructions
- Machine code – raw hexadecimal instructions representing binary; different instructions can have different byte lengths
 - Each instruction has an operation name (opcode)
 - Most instructions have arguments (operands)
 - \$[number] indicates a constant value or “immediate” operand
 - %[name] operand indicates a register – a storage location in the processor

```
00000000000001149 <main>:
1149:    f3 0f 1e fa
114d:    55
114e:    48 89 e5
1151:    be 01 00 00 00
1156:    48 8d 05 a7 0e 00 00
115d:    48 89 c7
1160:    b8 00 00 00 00
1165:    e8 e6 fe ff ff
116a:    be 02 00 00 00
116f:    48 8d 05 a7 0e 00 00
1176:    48 89 c7
1179:    b8 00 00 00 00
117e:    e8 cd fe ff ff
1183:    be 04 00 00 00

endbr64
push    %rbp
mov     %rsp,%rbp
mov     $0x1,%esi
lea     0xea7(%rip),%rax
mov     %rax,%rdi
mov     $0x0,%eax
call    1050 <printf@plt>
mov     $0x2,%esi
lea     0xea7(%rip),%rax
mov     %rax,%rdi
mov     $0x0,%eax
call    1050 <printf@plt>
mov     $0x4,%esi
```

Registers

- A register is a fast read/write memory on the CPU that can hold the values of variables
- They are separate from main memory
- Registers are like “scratch paper” for the processor – data being calculated or manipulated is moved to registers first and operations are performed on registers
- Registers also hold parameters and return values for functions
- Processor instructions mostly consist of moving data into/out of registers and performing arithmetic on them

x86-64 Registers

- Each register is 64-bit wide – the lower 32/16/8-bit portions can be selected using a pseudo-register name
- Some registers are designated for a specific purpose
 - `%rsp` as the stack pointer
 - `%rbp` is the base pointer
 - `%rax` for the return value from a function
- Other registers are general-purpose – but have a conventional use depending on whether caller-owned or callee-owned

x86-64 Registers (contd.)

- Registers used for the first six function arguments and return value are all callee-owned – the callee can freely use those registers, overwriting existing values without any precautions
- If %rax (callee-owned) holds a value the caller wants to retain, the caller must copy the value to a safe location before making a call
- Callee-owned registers are ideal for temporary use by the callee
- If the callee wants to use a caller-owned register, it must first preserve its value and restore it before exiting the call
- Caller-owned registers are used for the local state of the caller that needs to be preserved across further function calls

x86-64 Registers (contd.)

Register	Conventional use	Low 32-bits	Low 16-bits	Low 8-bits
%rax	Return value, callee-owned	%eax	%ax	%al
%rbx	Local variable, caller-owned	%ebx	%bx	%bl
%rcx	4th argument, callee-owned	%ecx	%cx	%cl
%rdx	3rd argument, callee-owned	%edx	%dx	%dl
%rsi	2nd argument, callee-owned	%esi	%si	%sil
%rdi	1st argument, callee-owned	%edi	%di	%dil
%rbp	Base/Frame pointer, caller-owned	%ebp	%bp	%bpl
%rsp	Stack pointer, caller-owned	%esp	%sp	%spl
%r8	5th argument, callee-owned	%r8d	%r8w	%r8b
%r9	6th argument, callee-owned	%r9d	%r9w	%r9b
%r10	Temporary use, callee-owned	%r10d	%r10w	%r10b
%r11	Temporary use, callee-owned	%r11d	%r11w	%r11b
%r12	Local variable, caller-owned	%r12d	%r12w	%r12b
%r13	Local variable, caller-owned	%r13d	%r13w	%r13b
%r14	Local variable, caller-owned	%r14d	%r14w	%r14b
%r15	Local variable, caller-owned	%r15d	%r15w	%r15b
%rip	Instruction pointer	%eip	%ip	-

The `mov` instruction and Addressing Modes

- The `mov` instruction copies bytes from one place to another – like the assignment operator in C

`mov src, dst`

- `src` and `dst` can be one of the following:
 - Immediate (constant value, only `src`)
 - Register
 - Memory location (at most one of `src` and `dst`)

Addressing Modes

- Immediate: `mov $0x10, dst` copy the value 0x10 to specified destination
- Register: `mov %rax, dst` OR `mov src, %rax`
Copy the value from register %rax to specified destination OR from specified source to %rax
- Direct: `mov 0x503d0, dst` OR `mov src, 0x503d0`
Copy the value at address 0x503d0 to specified destination OR from specified source to address 0x503d0
- Indirect: `mov (%rbx), dst` OR `mov src, (%rbx)`
Copy the value at the address stored in register %rbx to specified destination OR ...
- Indirect displacement: `mov 0x10(%rax), dst` OR `mov src, 0x10(%rax)`
Copy the value at address '0x10 plus what is stored in register %rax' to some destination OR ...
- Indirect scaled-index: `mov 0x8(%rsp, %rcx, 4), dst` OR `mov src, 0x8(%rsp, %rcx, 4)`
Copy the value at the address '0x8 + what is stored in register %rsp + what is stored in register %rcx*4' to some destination OR ...
- Any of the four parts can be omitted (default value 0 for the first three, 1 for the last (scale))

Assembly and C connection

- We see these indirect addressing paradigms in C as well
- Indirect addressing is like pointer arithmetic/dereferencing
- Equivalent C statements for assembly instructions:

• <code>mov \$0x0, %rdx</code>	it could be	<code>long x = 0</code>
• <code>mov %rdx, %rcx</code>	it could be	<code>long x = y;</code>
• <code>mov \$0x42, (%rdi)</code>	it could be	<code>*ptr = 0x42;</code>
• <code>mov (%rax, %rcx, 8), %rax</code>	it could be	<code>long x = arr[3];</code>

Variants of the `mov` instruction

- `mov` can take an optional suffix (b,w,l,q) that specifies the size of data to move:

`movb`

`movw`

`movl`

`movq`

- `mov` only updates the specific register bytes or memory locations indicated
 - Exception: **`movl`** writing to a register will also set the high-order 4 bytes to zero
- There are two specific `mov` instructions that can be used to copy a smaller source to a larger destination: **`movz`** and **`movs`**
- **`movz`** fills the remaining bytes with zeros
- **`movs`** fills the remaining bytes by sign-extending the most significant bit in the source
- The source must be from memory or a register, and the destination is a register

The **lea** instruction

- The **lea** instruction copies an “effective address” from one place to another

lea src, dst

- dst must be a register
- Unlike **mov**, which copies data at the address src to the dst, **lea** copies the value of src itself to dst
- The syntax for dst is the same as **mov** – the difference is how it handles src
- **mov** copies data at the address src to dest, **lea** copies the value of src to dst

mov vs. lea

Operands	mov interpretation	lea interpretation
<code>0x6(%rax), %rdx</code>	Go to the address (6 + what's in %rax), and copy data there into %rdx	Copy 6 + what's in %rax into %rdx
<code>(%rax, %rcx), %rdx</code>	Go to the address (what's in %rax + what's in %rcx) and copy data there into %rdx	Copy (what's in %rax + what's in %rcx) into %rdx
<code>(%rax, %rcx, 4), %rdx</code>	Go to the address ($\text{\%rax} + 4 * \text{\%rcx}$) and copy data there into %rdx	Copy ($\text{\%rax} + 4 * \text{\%rcx}$) into %rdx
<code>0x7(%rax, %rax, 8), %rdx</code>	Go to the address ($7 + \text{\%rax} + 8 * \text{\%rax}$) and copy data there into %rdx	Copy ($7 + \text{\%rax} + 8 * \text{\%rax}$) into %rdx